CLAUDE CERTIFIED ARCHITECT — PROFESSIONAL

Objective 10: Optimise Context Windows and Token Usage

Exam objective: Optimize context-window usage and token budgets through prioritization, conversation-history management, summarization/compaction, retrieval-based access to documents, tool-result optimization, subagent context isolation, and long-running session management.

Q1 **A customer-support agent has 50 registered tools and a large system prompt describing each one in detail, consuming a significant portion of the context window on every single turn, even though most conversations only ever use 3-4 tools. What is the BEST optimization?**

A. Keep all 50 tool definitions loaded on every turn to avoid any risk of a missing tool

B. **Dynamically load only the tool definitions relevant to the current conversation's intent, reducing the per-turn tool-definition footprint**

C. Remove the system prompt entirely to save tokens

D. Reduce the number of tools to exactly 3 for every conversation regardless of need

CORRECT ANSWER **B**

WHY Dynamically scoping tool definitions to the detected intent is the standard way to cut wasted context while preserving access to any tool actually needed. Always loading all 50 wastes tokens on every turn, removing the system prompt removes necessary guardrails/instructions, and a fixed arbitrary cap of 3 tools could break legitimate use cases needing a different tool.

Q2 **A financial advisory chatbot's conversation history grows to hundreds of turns over a multi-day engagement with a client. Token costs and context limits are becoming a problem. What is the BEST approach to manage this?**

A Always send the full, unabridged conversation history on every turn regardless of length

B Periodically summarize/compact older turns into a concise summary that preserves key decisions and facts, while keeping recent turns verbatim

C Truncate the conversation by deleting the oldest half with no summary at all

D Start a completely new conversation with no reference to prior history every time the limit is approached

CORRECT ANSWER **B**

WHY Summarization/compaction that preserves critical facts and decisions while dropping verbose older detail is the standard technique for managing long-running conversations within context limits. Sending everything unabridged doesn't scale, blind truncation risks losing important decisions, and starting over with zero history loses continuity the client expects.

Q3 **An architect is designing a research agent that uses a legal corpus of 50,000 documents. Early design sent entire relevant documents into context for every query. What is the BEST optimization for scaling this system?**

A Keep sending entire documents, but use the largest available context window model to accommodate it

B **Use retrieval (e.g., semantic search/RAG) to fetch only the most relevant passages or chunks for the specific query, rather than injecting entire documents**

C Randomly sample a few documents from the corpus each time

D Pre-load the entire 50,000-document corpus into every system prompt

CORRECT ANSWER **B**

WHY Retrieval of relevant passages rather than whole documents is the core technique for scaling beyond what fits (or is efficient) in a context window, regardless of model size. Relying on an ever-larger context window doesn't address cost/latency and won't scale indefinitely, random sampling ignores query relevance, and preloading the entire corpus is infeasible and wasteful.

Q4 **A manufacturing agent calls a diagnostics tool that returns a 15,000-token raw sensor log on every invocation, even though only the last 20 readings and any anomaly flags are typically relevant to the next reasoning step. What is the BEST tool-result optimization?**

A Pass the full 15,000-token log into context every time to avoid losing any information

B **Have the tool (or a post-processing step) pre-filter/summarize the result to just the relevant recent readings and anomaly flags before it enters the model's context**

C Stop using the diagnostics tool altogether

D Ask the model to request the log twice to make sure it was received

CORRECT ANSWER **B**

WHY Optimizing tool results—filtering or summarizing before they enter context—directly reduces wasted tokens while preserving what's actually needed for reasoning. Always passing the full log wastes context budget, removing the tool loses needed diagnostic capability, and requesting data twice compounds the waste.

Q5 **A multi-agent system has a "research" subagent that reads through dozens of long web pages to answer a narrow question, then reports back to a "writer" orchestrator agent. What is the BEST context-management pattern here?**

A Give the writer agent direct access to every raw web page the research agent read

B **Isolate the research subagent's large intermediate context (raw pages) within its own context window, and have it return only a concise, distilled answer/summary to the orchestrator**

C Merge both agents into a single context so nothing is isolated

D Have the orchestrator re-fetch and re-read all the same web pages independently

CORRECT ANSWER B

WHY Subagent context isolation lets the subagent do heavy, token-expensive work in its own window and return only the distilled result, keeping the orchestrator's context lean. Giving the orchestrator all raw pages defeats the purpose of delegation, merging contexts removes isolation benefits, and redundant re-fetching wastes resources without benefit.

Q6 **An e-commerce support agent's context window is filling up with repeated, verbose system reminders inserted on every turn (e.g., restating the full return policy each time). What is the BEST fix?**

A Keep repeating the full policy every turn to be safe

B **State the policy once in a stable, cacheable part of the system prompt (or reference it via retrieval only when relevant) rather than re-injecting it in full on every turn**

C Remove the return policy from the assistant's knowledge entirely

D Ask the user to restate the policy themselves each time

CORRECT ANSWER B

WHY Stable instructions belong once in a reusable, cacheable system prompt rather than being redundantly re-injected every turn, and context-specific policy detail can be retrieved only when the topic is actually relevant. Constant repetition wastes tokens, removing the policy loses necessary guardrails, and asking the user to restate it is impractical and unreliable.

Q7 **A telecom agent handling a very long troubleshooting session (100+ turns) must decide what to keep in context as the conversation approaches the window limit. Which prioritization strategy is MOST appropriate?**

A Keep the oldest turns and drop the most recent ones

B **Prioritize keeping the most recent turns (immediate context) plus a compact summary of key facts/decisions from earlier in the conversation, dropping low-value filler (e.g., small talk, resolved side-issues)**

C Keep every turn exactly as-is regardless of relevance or recency

D Keep only turns that mention the word "error"

CORRECT ANSWER **B**

WHY The standard approach balances recency (most relevant to the current state) with a condensed record of important earlier facts/decisions, while discarding low-value content. Dropping recent turns loses the active thread of the conversation, keeping everything doesn't scale, and keyword-based filtering is too brittle to reliably preserve important context.

Q8 **An architect is reviewing a long-running autonomous coding agent that has been operating for several hours across many tool calls, accumulating a large interaction history. The agent's performance appears to be degrading due to context bloat. What is the BEST remediation?**

A Let the agent continue indefinitely with the full accumulated history

B **Periodically checkpoint/compact progress (summarizing completed steps and key decisions) and prune low-value intermediate tool output, preserving only what is needed to continue the task correctly**

C Restart the agent from scratch with zero memory of prior progress

D Increase the verbosity of logging sent into context to help the agent "remember better"

CORRECT ANSWER B

WHY Periodic compaction/checkpointing of long-running agent sessions—summarizing progress and pruning stale intermediate detail—is the standard mitigation for context bloat degrading performance over long sessions. Unbounded accumulation worsens the problem, a full restart discards legitimate progress, and adding more verbose logging makes the bloat problem worse, not better.

Q9 **A healthcare agent's context budget is tight, and the architect must choose what to prioritize when both the full patient history and the current clinical question compete for space. What is the BEST prioritization principle?**

A. Always prioritize the oldest historical records over the current question

B. Prioritize information directly relevant to answering the current clinical question, retrieving only the most pertinent parts of history rather than the entire record

C. Include the entire patient record every time regardless of relevance to the current question

D. Discard all patient history to save space and rely only on the current question

CORRECT ANSWER B

WHY Context prioritization should favor relevance to the task at hand, using retrieval to surface only the pertinent historical details rather than everything or nothing. Prioritizing old records indiscriminately or including the entire record wastes context on irrelevant detail, while discarding all history risks missing clinically relevant facts.

Q10 **A SaaS company's agent occasionally needs to review very large log files (100K+ tokens) that exceed what should reasonably be placed in the main conversation context. What is the BEST architectural pattern?**

A Paste the entire log file into the main conversation context regardless of size

B Delegate log analysis to an isolated subagent (or a retrieval/chunked-processing step) that processes the large file separately and returns only a concise, relevant summary or specific findings to the main context

C Ask the user to manually read the log and summarize it themselves

D Truncate the log file at a fixed byte count with no regard for what's cut off

CORRECT ANSWER B

WHY Isolating large-document processing in a subagent or chunked pipeline, returning only distilled findings, is the correct pattern for keeping the main context lean while still handling oversized inputs. Pasting the whole file blows the budget, relying on manual user effort isn't scalable automation, and naive fixed-size truncation risks cutting off the most relevant portion.

Q11 **An architect notices that a multi-turn negotiation agent keeps re-sending the entire product catalog (5,000 tokens) in every turn "just in case" the user asks about a different product. What is the BEST fix?**

A Continue sending the full catalog every turn since partial information could cause errors

B **Retrieve and inject only the specific product details relevant to what the user is currently discussing, re-retrieving as the topic shifts**

C Remove the product catalog from the system entirely

D Ask the user to specify the exact SKU before any response is given

CORRECT ANSWER B

WHY Targeted, query-driven retrieval of only the relevant product details—re-fetched as the conversation topic shifts—avoids the waste of resending an entire catalog every turn. Always sending everything is wasteful, removing the catalog eliminates needed capability, and demanding exact SKUs upfront creates unnecessary friction for the user.

Q12 **A government document-processing pipeline uses an agent that calls a search tool which can return up to 50 results per query, each with full document text. What is the BEST way to optimize this tool's context footprint?**

A Always request and inject all 50 full documents for every search

B Limit results to a smaller top-N most relevant set and return only titles/snippets by default, fetching full text only for documents the agent specifically needs to examine further

C Disable the search tool and manually curate documents instead

D Request 50 results but truncate the context window size instead

CORRECT ANSWER B

WHY Returning a smaller, ranked set of snippets by default and fetching full text only on demand is the standard tool-result optimization pattern that balances recall with context efficiency. Injecting all 50 full documents floods the context, disabling the tool removes valuable automation, and shrinking the model's context window doesn't address the tool's inefficient result shape.

Q13 **An architect designing a long-running customer-onboarding agent wants to ensure that critical commitments made early in the conversation (e.g., a promised discount) are never lost even after many rounds of summarization. What is the BEST safeguard?**

A Rely entirely on generic summarization and hope important details survive

B **Explicitly tag and persist critical facts/decisions (e.g., in a structured "key facts" memory store or protected summary section) so they survive compaction regardless of generic summarization choices**

C Avoid summarizing at all, no matter how long the conversation becomes

D Ask the customer to repeat the discount commitment themselves each time

CORRECT ANSWER B

WHY Structured persistence of explicitly flagged critical facts protects them from being inadvertently dropped during generic summarization/compaction. Relying purely on generic summarization risks losing important specifics, refusing to summarize at all doesn't scale for long conversations, and expecting the customer to repeat commitments is both unreliable and poor user experience.

Q14 **A logistics company's agent orchestrates five subagents, each of which independently reads large shipment manifests. An architect is evaluating whether subagents should share one combined context or operate in isolated contexts. What is the BEST guidance?**

A Always combine all subagents into a single shared context to simplify the system

B **Keep each subagent's large intermediate working context isolated to its own window, and pass only concise, structured results between subagents/orchestrator to prevent cross-contamination and token bloat**

C Isolate subagents completely with no information ever passed between them

D Randomly decide per run whether to isolate or share context

CORRECT ANSWER B

WHY Isolating each subagent's heavy intermediate context while passing only distilled, structured results between agents prevents unnecessary token bloat and cross-contamination while still enabling coordination. Fully combining all contexts reintroduces the bloat problem, total isolation with no information passed breaks coordination, and random policy decisions are not a sound architecture.

Q15 An architect must decide how to handle a user who pastes an entire 30-page contract into the chat and asks a single narrow question about one clause. What is the MOST context-efficient approach?

A Send the entire 30-page contract into the model's context for every follow-up question in the conversation, even unrelated ones

B Process the contract once (e.g., chunk and index it), answer the specific question using only the relevant clause(s), and retrieve additional sections on demand for later questions rather than resending the whole document every turn

C Refuse to process the contract because it is too long

D Ask the user to manually extract and paste just the relevant clause themselves

CORRECT ANSWER B

WHY One-time processing/indexing followed by targeted retrieval per question is the efficient pattern for handling long pasted documents across a multi-turn conversation. Resending the full document every turn wastes enormous context, refusing to process it fails the user's legitimate request, and requiring the user to manually extract clauses defeats the purpose of using the assistant.

Q16 **An architect loads an entire 80,000-token internal wiki into a research agent's context "just in case," even though most queries only need a small fraction of it. Users report that the agent sometimes fails to use information that is clearly present in the middle of the loaded material, while information near the beginning and end of the context still gets used correctly. What does this symptom MOST likely indicate?**

A A hardware failure in the model-serving infrastructure

B **The context window is being treated as storage rather than working memory; recall degrades as token count grows, and it degrades unevenly, with content in the middle of a long loaded collection becoming less likely to be used even though nothing has technically failed or errored**

C The wiki content must be corrupted, since the model would otherwise use all of it equally

D This is expected and has no relationship to how much content was loaded into the window

CORRECT ANSWER B

WHY This is the documented "lost in the middle"/working-memory-cliff phenomenon: adding more context is not automatically better, accuracy and recall degrade as token count grows, and the degradation is silent and uneven, with middle content becoming unreliable first while the ends still work. It is not a hardware issue or data corruption, and reducing what enters the window per request—not re-ordering or re-emphasizing content—is the real fix.

Q17 An architect is building a service that generates very long reports and wants to understand the failure behavior when a request approaches the context window limit. Which statement correctly distinguishes the two ways a request can exceed the window?

A Both input-only overflow and input-plus-output overflow always return the same generic 400 error with no distinction

B **If the input alone exceeds the window, the request fails immediately with a 400 invalid request error; if input plus max_tokens would exceed the window, some newer models instead accept the request, run it, and stop generation with a `model_context_window_exceeded` stop reason once the limit is actually reached, so a caller that checks only HTTP status may mistake a truncated response for a complete one**

C Exceeding the window is never possible because models automatically truncate input silently with no indication at all

D Only output overflow is ever rejected; input overflow is always silently ignored and processed anyway

CORRECT ANSWER **B**

WHY Documented behavior distinguishes the two failure modes precisely: input-only overflow is rejected up front with a 400 error, while input-plus-max_tokens overflow on newer models is accepted and only stops mid-generation with a specific stop reason, which a caller checking only for HTTP errors could misinterpret as a complete answer. The other options collapse or invert this distinction.

Q18 An architect configures automatic compaction for a long-running agent and writes custom `instructions` to additionally capture a specific "open commitments" list whenever compaction runs. After deployment, the team notices the agent sometimes loses track of its next steps and overall task state, which did not happen before the custom instructions were added. What is the MOST likely cause?

A Custom compaction instructions supplement the default summarization prompt automatically, so this cannot be the cause

B Custom instructions supplied through the `instructions` parameter REPLACE the default summarization prompt entirely rather than adding to it, so the team's new instruction silently discarded the default behavior of recording state, next steps, and learnings needed to continue the task

C Compaction never removes any prior content, so state loss must come from an unrelated bug

D The architect should have used context editing instead, since compaction and context editing are functionally identical

CORRECT ANSWER B

WHY Custom compaction instructions replace the default summarization prompt completely rather than supplementing it, so adding one new instruction without also including the default guidance to record state, next steps, and learnings silently drops that default behavior, explaining the regression. Custom instructions do not merge with the default, compaction does drop prior content blocks before the compaction point, and compaction/context editing are distinct mechanisms (summarizing vs. selectively removing) rather than identical. --